Code and Functional Coverages

Objectives

- To learn about code coverage
- To have an idea about functional coverages
- To learn the types of code coverages
- To learn the types of Functional Coverages

Session Topics

- Code Coverage
- Types of Code coverage
- Functional Coverage
- Types of Functional Coverage
- Merits and demerits of code and functional coverages

Introduction to Code Coverage

- Code coverage is used to measure the efficiency of verification implementation.
- It provides a quantitative measurement of the testing space.
- It describes the degree to which the source code of a DUT has been tested. It is also referred as structural coverage.

Introduction to Code Coverage

Code coverage answers the questions like

Have all the branches in " Case ", "if" have been entered?

Have all the conditions in "if", "case" statement is simulated?

Have all the variables have been toggled?

Have all the statements of the RTL code have been exercised?

Have all the states in the FSM has been entered and all the legal transitions exercised?

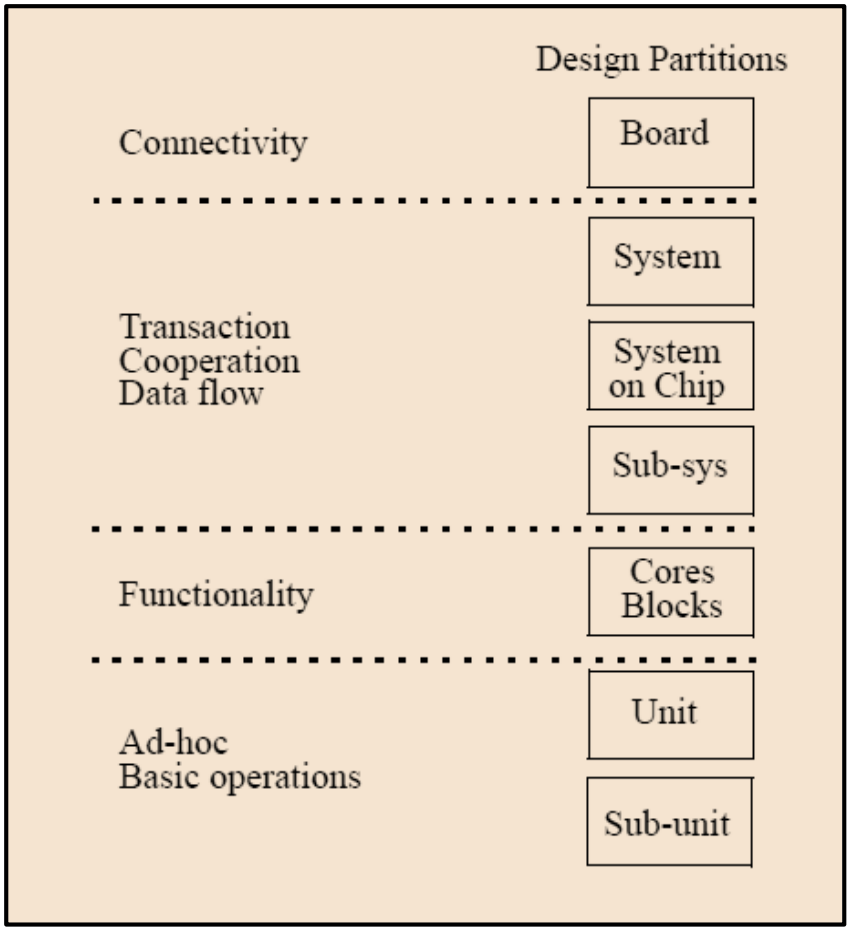
Have all the paths within a block have been exercised?

Performance Measure

- By applying code coverage analysis techniques to hardware description languages, verification efficiency was improved by enabling a verification engineer to isolate areas of un-tested HDL code.
- The verification engineer examine a coverage report, seeks out the low values and understands why that particular code hasn't been tested fully and writes more tests or directs randomness to cover the untested areas where there may be a possibility of bug hiding.
- It does not require any additional coding to get code coverage, tool dose everything.

Levels of Verification

- Each level of Verification will be suited for a specific objective
  - Lower levels have better controllability and visibility
- Block level Verification helps designs to be verified independently and in parallel
- System Level Verification focuses more on interactions



Performance Measure

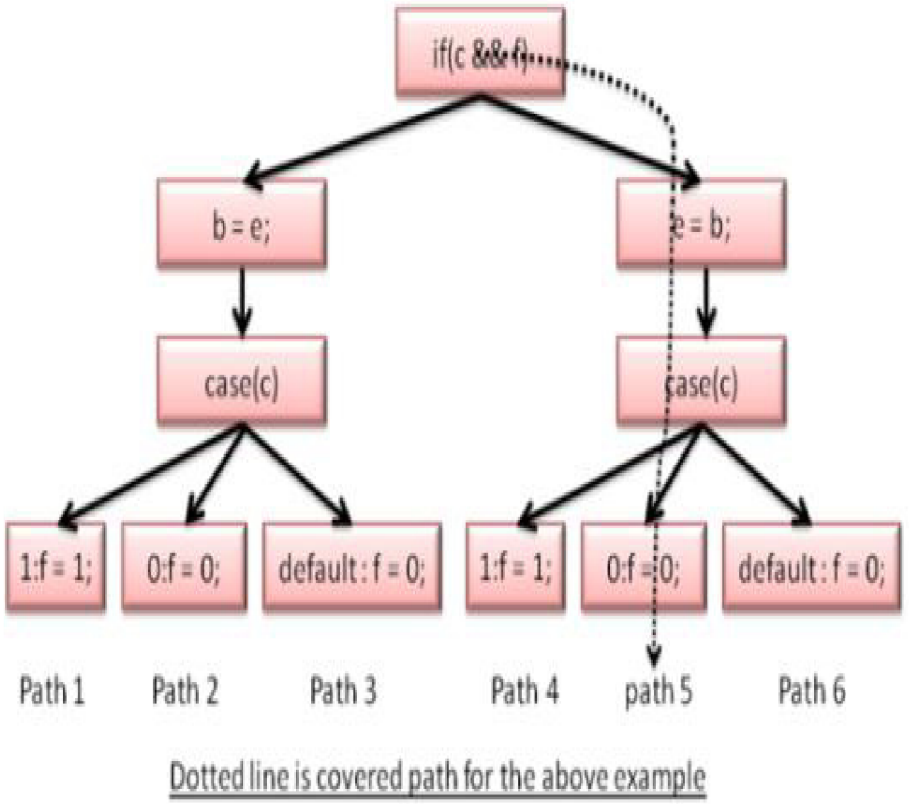
- In unit level verification, a module by module is verified in its own test environment to prove that the logic, control, and data paths are functionally correct.
- The goal of module level verification is to ensure that the component/unit being tested conforms to its specifications and is ready to be integrated with other subcomponents of the product.
- Code coverage becomes a criterion for finishing unit level testing as it needs to verify every feature of component/unit. In sub-system level /system level, the goal is to ensure that the interfaces among the units are correct and the units work together to execute the functionality correctly.
- In sub system level /system level testing, code coverage may not be use full as the verification is not targeted at all the features of the unit.

TYPES OF CODE COVERAGE

- Statement coverage /line coverage
- Block/segment coverage
- Conditional coverage
- Branch coverage
- Toggle coverage
- path coverage
- Fsm coverage

Code Coverage Example

```
2       module dut();
3       reg a,b,c,d,e,f;
4
5       initial
6       begin
7           #5 a = 0;
8           #5 a = 1;
9       end
10
11      always @(posedge a)
12      begin
13          c = b && a;
14          if(c && f)
15            b = e;
16          else
17e = b;
18
19          case(c)
20            1:f = 1;
21            0:f = 0;
22            default : f = 0;
23          endcase
24
25      end
26 endmodule
```



STATEMENT COVERAGE

- Statement coverage, also known as line coverage is the easiest understandable type of coverage. This is required to be 100% for every project.
- From N lines of code and according to the applied stimulus how many statements (lines) are covered in the simulation is measured by statement coverage.
- If a DUT is 10 lines long and 8 lines of them were exercised in a test run, then the DUT has line coverage of 80%. Line coverage includes continuous assignment statements, Individual procedural statements, Procedural statement blocks, Procedural statement block types, Conditional statement and Branches for conditional statements.

STATEMENT COVERAGE

- It considers only the executable statements and statements which are not executable like module, endmodule, comments, timescale etc are not covered.

There are total 12 statements at lines 5,7,8,11,13,14,15,17,19,20,21,22

Covered 9 statements. They are at lines
5,7,8,11,13,14,17,19,22

Uncovered 3 statements. They are at line
15,20,22

Coverage percentage: 75.00 (9/12)

BLOCK COVERAGE

- The nature of the statement and block coverage looks somewhat same.
- The difference is that block coverage considers branched blocks of if/else, case branches, wait, while, for etc.
- Analysis of block coverage reveals the dead code in RTL.

There are total 9 blocks at lines
5,7,8,11,15,17,20,21,22

Covered 6 blocks. They are at lines
5,7,8,11,17,22

Uncovered 3 blocks. They are at line
15,20,22

Coverage percentage: 66.67 (6/9)

CONDITIONAL COVERAGE

- Conditional coverage also called as expression coverage, will reveals how the variables or sub-expressions in conditional statements are evaluated. Expressions with logical operators are only considered.
- The downside is that the conditional coverage measure doesn't take into consideration how the Boolean value was gotten from the conditions.
- Conditional coverage is the ratio of no. of cases checked to the total no. of cases present. Suppose one expression having Boolean expression like AND or OR, so entries which is given to that expression to the total possibilities is called expression coverage.

CONDITIONAL COVERAGE

- Conditional coverage report of the previous example:

  At LINE 13

  Combinations of STATEMENT c = (b && a)

  B = 0 and a = 0 is Covered
  B = 0 and a = 1 is Covered
  B = 1 and a = 0 is Not Covered
  b = 1 and a = 1 is Not Covered

  At LINE 14 combinations of STATEMENT if ((c && f)) C = 0 and f = 0 is Covered
  C = 0 and f = 1 is Not Covered
  C = 1 and f = 0 is Not Covered
  C = 1 and f = 1 is Not Covered

  Total possible combinations: 8

  Total combinations executed: 3

BRANCH COVERAGE

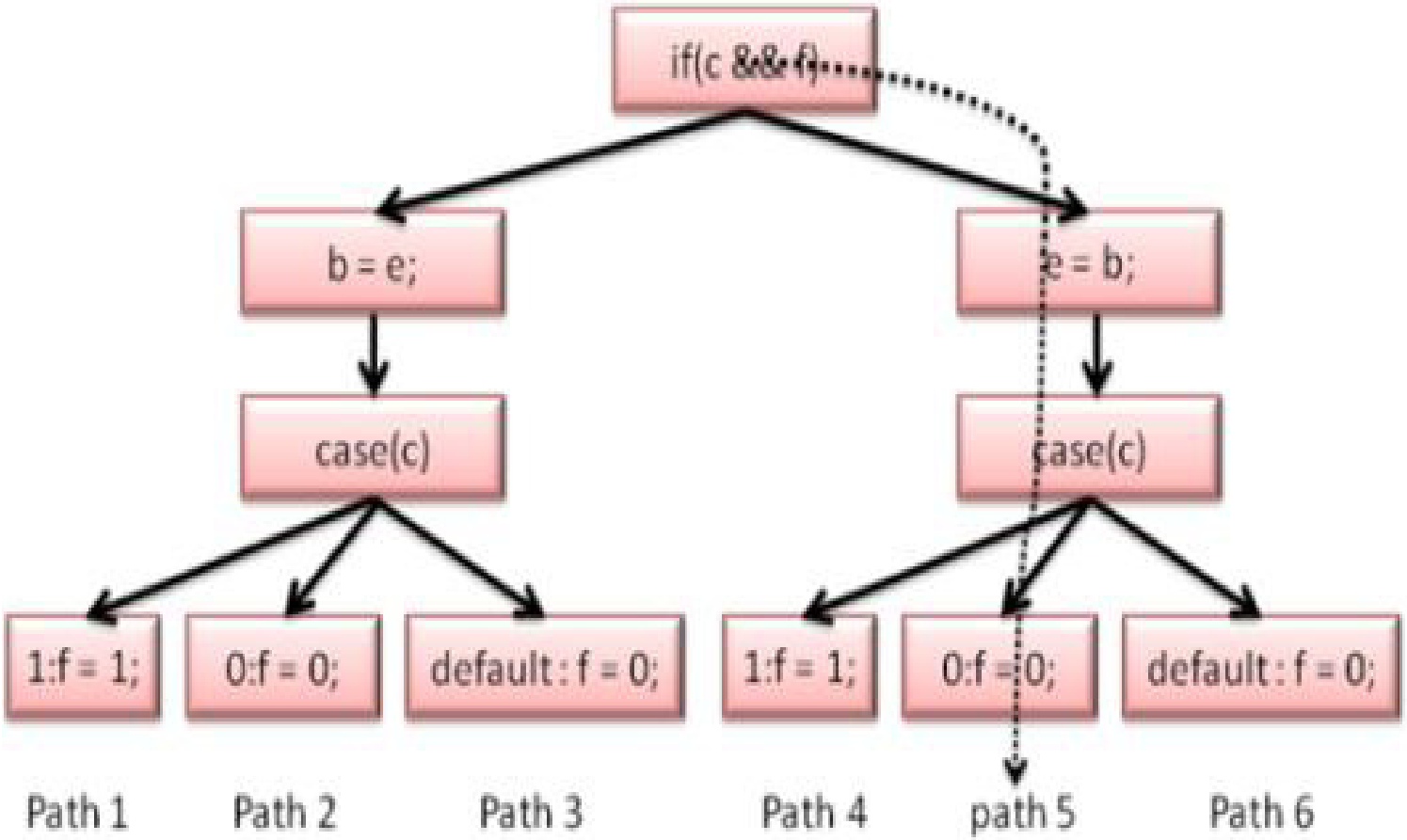
- Branch coverage which is also called as Decision coverage report s the true or false of the conditions like if-else, case and the ternary operator (? :) statements.
- For an "if" statement, decision coverage will report whether the "if" statement is evaluated in both true and false cases, even if "else" statement doesn't exist.

Branch coverage report of the example:

At line 15 branch b = e; not covered
At line 17 branch e = b; covered
At line 20 branch 1: f = 1; not covered
At line 21 branch 0: f = 0; covered
At line 22 branch default: f = 0; not covered

Coverage percentage: 40.00 (2/5)

PATH COVERAGE



Dotted line is covered path for the above example

PATH COVERAGE

- Path coverage represents yet another interesting measure. Due to conditional statements like if-else, case in the design different path is created which diverts the flow of stimulus to the specific path.
- Path coverage is considered to be more complete than branch coverage because it can detect the errors related to the sequence of operations.
- As mentioned in the above figure path will be decided according to the if-else statement According to the applied stimulus the condition which is satisfied only under those expressions will execute, the path will be diverted according to that.
- Path coverage is possible in always and function blocks . Path created by more than one block is not covered.

PATH COVERAGE

- Path coverage report of the example:

Path 1 : 15,20 Not Covered
Path 2 : 15,21 Not Covered
Path 3: 15,22 Not Covered
Path 4: 17,20 Not Covered
Path 5 : 17,21 Covered
Path 6 : 17,22 Not Covered

Total possible paths : 6
Total covered path : 1
Path coverage Percentage : 16.67 (1/6)

TOGGLE COVERAGE

- It makes assures that how many times variables and nets toggled? Toggle coverage could be as simple as the ratio of nodes toggled to the total number of nodes.

X or Z --> 1 or H
X or Z --> 0 or L
1 or H --> X or Z
0 or L --> X or Z

TOGGLE COVERAGE

- Above example shows the signal changes from one level to another. All types of transitions mentioned above are not interested. Only 1->0 and 0->1 are important.
- Toggle coverage will show which signal did not change the state. Toggle coverage will not consider zero-delay glitches. This is very useful in gate level simulation.

Toggle coverage report of the example:

Name	Toggled	1->0	0->1
a	Yes	No	Yes
b	No	No	No
c	No	No	No
d	No	No	No
e	No	No	No
f	No	No	No

FSM COVERAGE

- It is the most complex type of code coverage, because it works on the behavior of the design.
- Using Finite state machine coverage, all bugs related to finite state machine design can be found. In this coverage we look for how many times states are visited, transited and how many sequence are covered in a Finite state machine.
- It will count the no. of transition from one state to another and it will compare it with other total no. of transition. Total no. of transition is nothing but all possible no. of transition which is present in the finite state machine. Possible transition = no. of states * no. of inputs.

Example

```
module fsm (clk, reset, in);
input clk, reset, in;   reg
[1:0] state;

    parameter s1 = 2'b00; parameter s2 = 2'b01;
    parameter s3 = 2'b10; parameter s4 = 2'b11;

    always @(posedge clk or posedge reset)
    begin
      if (reset) state <= s1;
      else case (state)
     s1:if (in == 1'b1) state <= s2;
        else state <= s3;
     s2: state <= s4;
     s3: state <= s4;
     s4: state <= s1;
           endcase
    end
endmodule
```

TestBench

```
module testbench();
    reg clk,reset,in;

    fsm dut(clk,reset,in);

    initial
      forever #5 clk = ~clk;

    initial
    begin
      clk =0;in = 0;
      #2 reset = 0;#2 reset = 1;
      #21 reset = 0;#9 in = 0;
      #9 in = 1;#10 $finish;
    end

endmodule
```

FSM coverage report

- FSM coverage report for the above example:

```
// state coverage results
s1                                  | Covered
s2                                  | Not Covered
s3                                  | Covered
s4                                  | Covered
// state transition coverage results
s1->s2                                | Not Covered
s1->s3                                | Covered
s2->s1                                | Not Covered
s2->s4                                | Not Covered
s3->s1                                | Not Covered
s3->s4                                | Covered
s4->s1                                | Covered
```

MAKE YOUR GOAL 100 PERCENT CODE COVERAGE NOTHING LESS

- Never set your goal to anything less than 100% code coverage. Anything less than 100% is a slippery slope. If you set your goal to 98% , may be the most important feature like reset of the system may be in the untested part of 2%.
- If the verification engineer sets the code coverage goal to 95% to facilitate the 5% the unused untestable legacy code, there are chances that the unused legacy code gets executed and the 5% holes may be in the important code.
- 100% code coverage provides advantages not only in reducing the bug count but also in making it easier to make significant changes to existing code base to remove uncover able areas like the unused legacy blocks in RTL code.

Dont Be Fooled By The Code Coverage Report

- Highly covered code isn't necessarilyfree of defects, although it's certainly less likely to contain them. By definition, code coverage is limited to the design code. It doesn't know anything about what design supposed to do.
- Even if a feature is not implemented in design, code coverage can report 100% coverage.
- It is also impossible to determine whether we tested all possible values of a feature using code coverage
- Code coverage is unable to tell much about how well you have covered your logic -- only whether you've executed each line/block etc at least once.

Dont Be Fooled By The Code Coverage Report

- Code coverage does not provide information about your test bench randomization quality and it does not report what caused the line execution/state transition etc.
- Analysis of code coverage require knowledge of design to find which features are not verified which is time consuming and out of scope of verification engineer.
- If the analysis is done at higher level of abstraction, it would be easier for the test writer to identify the missed serious which is not possible by code coverage.
- So if the code coverage is less than 100%, it means there is more work to do, if it is 100%, it doesn't mean that the verification is complete.

When To Stop Testing?

- It's getting harder to figure out when to stop testing as the complexity of the protocol is increasing.
- In directed test environment, for each point mentioned in test plan, there will be a separate test case file.
- So if there are 100 points in test plan, then the engineer has to write 100 test case files.
- After writing and executing the 100 test case files, we can say that "all the points in test plan are verified" and we can stop testing.

FUNCTIONAL COVERAGE

- In constraint random verification all the features are generated randomly. Verification engineer need a mechanism to know the information about the verified features of DUT.
- SystemVerilog provides a mechanism to know the untested feature using functional coverage.
- Functional Coverage is "instrumentation" that is manually added to the TestBench.
- This is a better approach than counting testcases.
- Functional coverage is better than code coverage where the code coverage reports what was exercised rather than what was tested

Functional Coverage Answers

- Have all the packets length between 64 to 1518 are used?

  Did the DUT got exercised with alternate packets with good and bad crc?

  Did the monitor observe that the result comes with 4 clock cycles after read operation?

  Did the fifos are filled completely?

  Did the fifo takes care of empty and full?

Functional Coverage Advantages

- Functional coverage helps to determine how much of your specification was covered.
- Functional coverage qualifies the test benches. Considered as stopping criteria for unit level verification.
- Gives feedback about the untested features.
- Gives the information about the redundant tests which consume valuable cycle.
- Guides to reach the goals earlier based on grading.

Verification Approaches

- Black-Box
  - ☺ Verification without knowledge of design implementation.
  - ☹ Lack of visibility and observability.
  - ☺ Tests are independent of implementation.
  - ☹ Impractical in todays designs.
- White Box
  - ☺ Intimate knowledge of design implementation
  - ☺ Full visibility and observability
  - ☹ Tests are tied to a specific implementation
- Grey Box
  - ☺ Compromise between above approaches